Empório da Música — AI Customer Service Agent

Technical project documentation

Python conversational agent for the fictional Empório da Música musical instrument store in Campo Grande, Brazil. It combines OpenAI Function Calling for structured operational data with a lightweight lexical RAG layer for store policies.

Features

- Product search by name, description, category, and maximum price.
- Deterministic price, inventory, promotion, and order lookups from CSV files.
- Order lookup protected by account-holder name validation.
- Store address, contacts, opening hours, and payment methods.
- Policy retrieval for exchanges, returns, shipping, warranty, and customer service.
- Session message history, error handling, and human-support escalation.

Architecture

```
Customer (CLI)
    |
    v
OpenAI Chat Completions + Function Calling
    |
    +-- search_products ------> Product CSVs, categories, promotions
    +-- get_order_status -----> Orders, customers, order items
    +-- store_information ----> Official store information
    +-- search_store_policies -> Lexical retrieval from the policy manual
```

Structured data is queried through deterministic functions so the model cannot invent prices, inventory, or order status. Policy questions use lexical retrieval because the supplied manual is small and textual. A production version could add embeddings, hybrid search, reranking, citations, and a transactional database.

Technology and Configuration

- Python 3.10+
- OpenAI Python SDK and Chat Completions with Function Calling
- Default model: `gpt-4o-mini`, configurable through `OPENAI_MODEL`
- API key loaded from `OPENAI_API_KEY`; secrets are never stored in source code
- `python-dotenv` for local `.env` configuration

How to Run

Prerequisites

- Python 3.10 or higher
- An OpenAI API key with access to the configured model

Installation

```
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
```

Configuration

Copy `.env.example` to `.env` and add your key:

```
OPENAI_API_KEY=your_key
OPENAI_MODEL=gpt-4o-mini
```

Never commit `.env` or expose the API key.

Start

```
python app.py
```

CLI commands: `/clear` resets the session history and `/exit` exits the application. The assistant speaks English.

Data and Privacy

The challenge data is fictional. Order lookup requires both an order number and the account holder's full name. The prototype does not persist conversations between executions and does not provide real payment, shipping, WhatsApp, or CRM integrations.

Limitations and Next Steps

The lexical RAG layer may miss synonyms, CSV files are static, and authentication is intentionally a prototype. A production implementation should add stronger authentication such as OTP, attempt limiting, audit logs, database-backed updates, automated evaluations, tracing, and retention controls.

Project Structure

```
emporio-musica-ai-agent/
■■■ app.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ data/
■   ■■■ products.csv
■   ■■■ promotions.csv
■   ■■■ customers.csv
■   ■■■ categories.csv
■   ■■■ order_items.csv
■   ■■■ orders.csv
■   ■■■ store_policies.txt
■■■ examples/
    ■■■ 01_catalog.md
    ■■■ 02_returns.md
    ■■■ 03_order.md
    ■■■ 04_hours.md
    ■■■ 05_out_of_scope.md
```

Coding Assistant Transparency

ChatGPT supported requirement interpretation, architecture planning, implementation review, and documentation organization. Business rules, limitations, and data-handling decisions are documented for technical review.